

1. A visualização de dados é o primeiro passo porque ela revela padrões que nem sempre aparecem em tabelas. Um histograma pode mostrar assimetria, dispersão, multimodalidade e outliers. Uma matriz de dispersão pode revelar correlação entre variáveis, agrupamentos, relações não lineares e separação entre classes.

2. Uma distribuição uniforme tem frequências aproximadamente constantes em um intervalo. Uma distribuição normal concentra a maioria dos valores perto da média e diminui nas extremidades, formando o formato de sino. Um histograma é ideal para identificar essas formas porque mostra visualmente a frequência por faixa de valores.

3. Duas estratégias comuns para valores ausentes em uma coluna numérica são:
Remover as linhas com valores ausentes. Vantagem: simples e rápido. Desvantagem: pode reduzir muito o conjunto de dados e introduzir viés se os faltantes não forem aleatórios.
Preencher os valores ausentes com média ou mediana. Vantagem: preserva mais linhas. Desvantagem: pode distorcer a distribuição e reduzir a variabilidade real dos dados.

4. A função `try_or_none` tenta converter cada string em inteiro e retorna `None` quando não consegue.

```
def try_or_none(func, value):  
    try:  
        return func(value)  
    except Exception:  
        return None  
  
def converter_para_inteiros(valores):  
    return [try_or_none(int, valor) for valor in valores]
```

5. O scaling é crucial em algoritmos baseados em distância porque variáveis em escalas maiores passam a dominar o cálculo. Em k-NN ou clustering, isso pode distorcer as vizinhanças e os grupos formados. Padronizar ou normalizar coloca as variáveis em uma escala comparável.

6. O `StandardScaler` transforma as variáveis para terem média zero e desvio padrão um, o que coloca altura e peso na mesma escala e evita que uma variável pese mais do que a outra.

```
def standardize_features(data):  
    scaler = StandardScaler()  
    return scaler.fit_transform(data)
```

7. A biblioteca tqdm mostra uma barra de progresso durante tarefas demoradas, ajudando a acompanhar o andamento da execução.

```
def simular_tarefa_demorada(iterations=100, delay_seconds=0.01):  
    for _ in tqdm(range(iterations), desc="Processando"):  
        sleep(delay_seconds)
```

8. Quando você precisa do item e do índice, basta combinar tqdm com enumerate.

```
def iterar_com_indice(items):  
    for index, item in tqdm(enumerate(items), total=len(items),  
desc="Iterando"):  
        print(index, item)
```

9. O argumento leave=False faz a barra de progresso sumir ao terminar o loop, o que deixa a saída mais limpa em loops aninhados.

10. Para integrar tqdm com apply no Pandas, use tqdm.pandas() e depois progress_apply para acompanhar o processamento linha por linha.

```
def aplicar_com_tqdm(df, func, result_column):  
    tqdm.pandas()  
    df[result_column] = df.progress_apply(func, axis=1)  
    return df
```